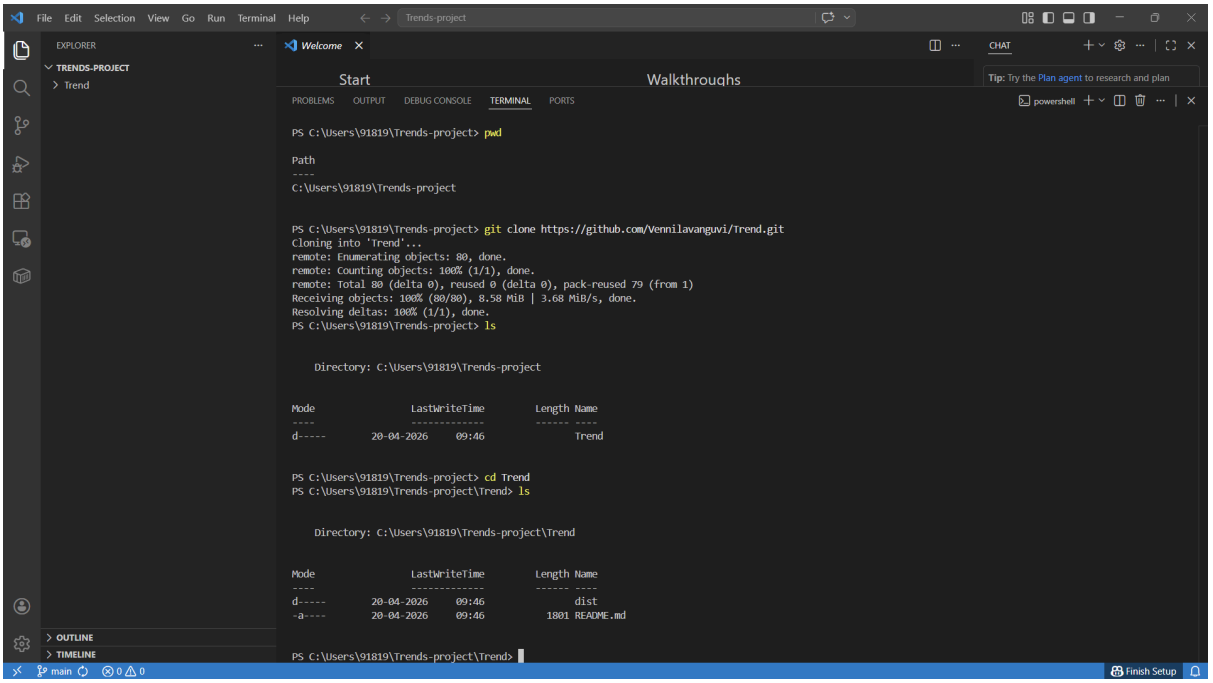
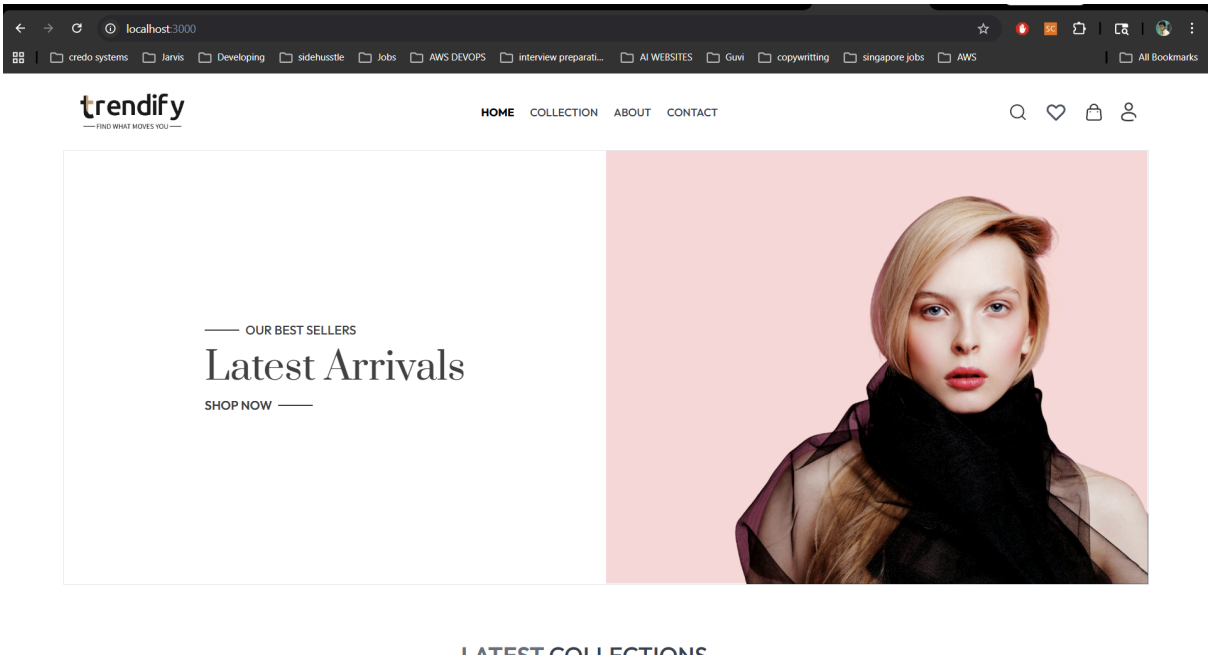
Application:

Clone the below mentioned repository and deploy the application (Run application in port 3000).

Repo URL : https://github.com/Vennilavanguvi/Trend.git



After the cloning part is done i am going to run manually in my local machine in the port 3000 npx serve - l 3000 dist



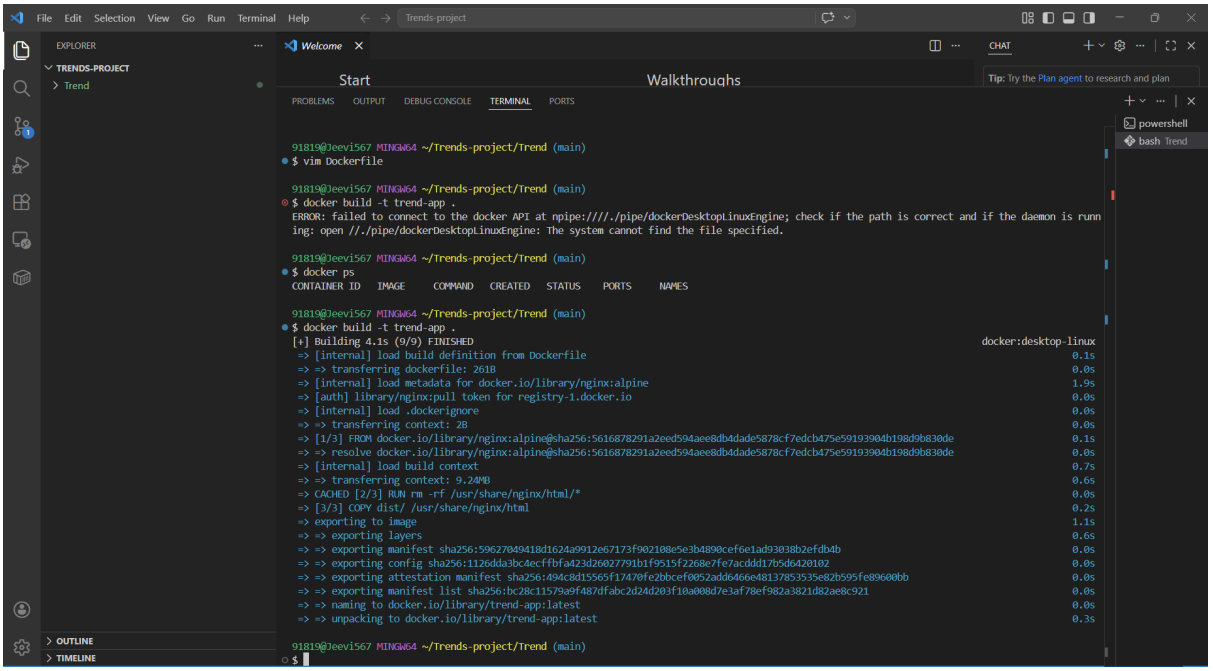
Docker:

- Dockerize the application by creating Dockerfile

```
FROM nginx:alpine

# Remove default nginx content
RUN rm -rf /usr/share/nginx/html/*

# Copy your built app
COPY dist/ /usr/share/nginx/html

# Expose nginx port
EXPOSE 80

# Start nginx
CMD ["nginx", "-g", "daemon off;"]
```

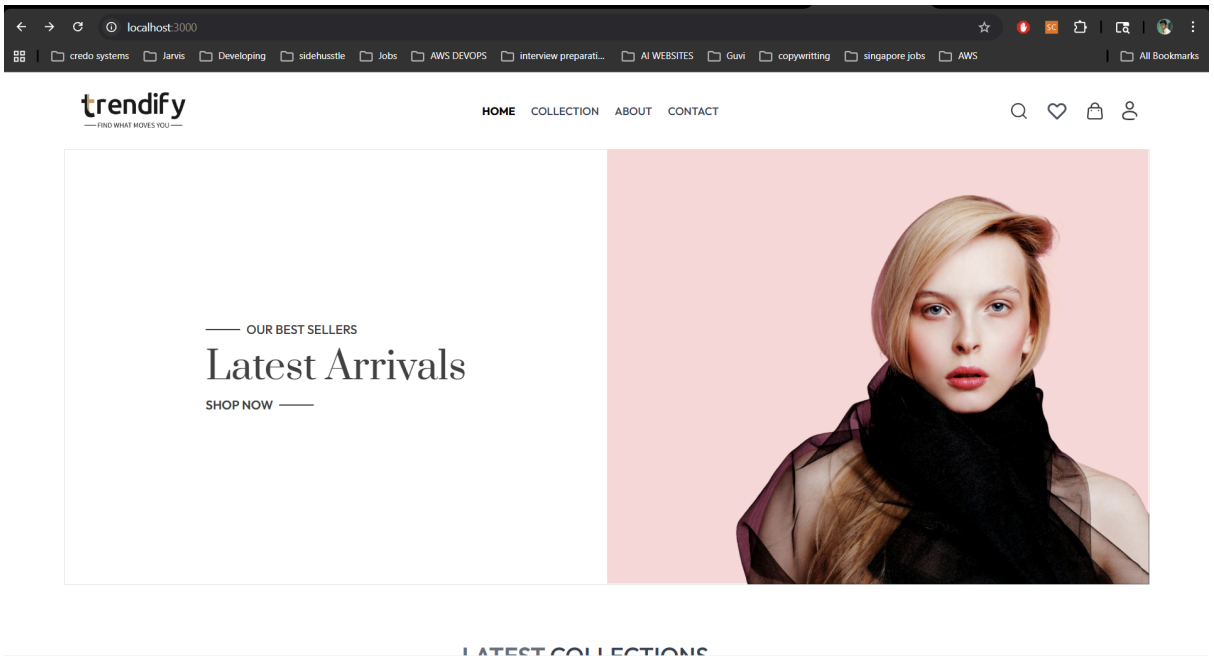
Build an application and check output using docker image.

At next I am going to run this docker file using this command

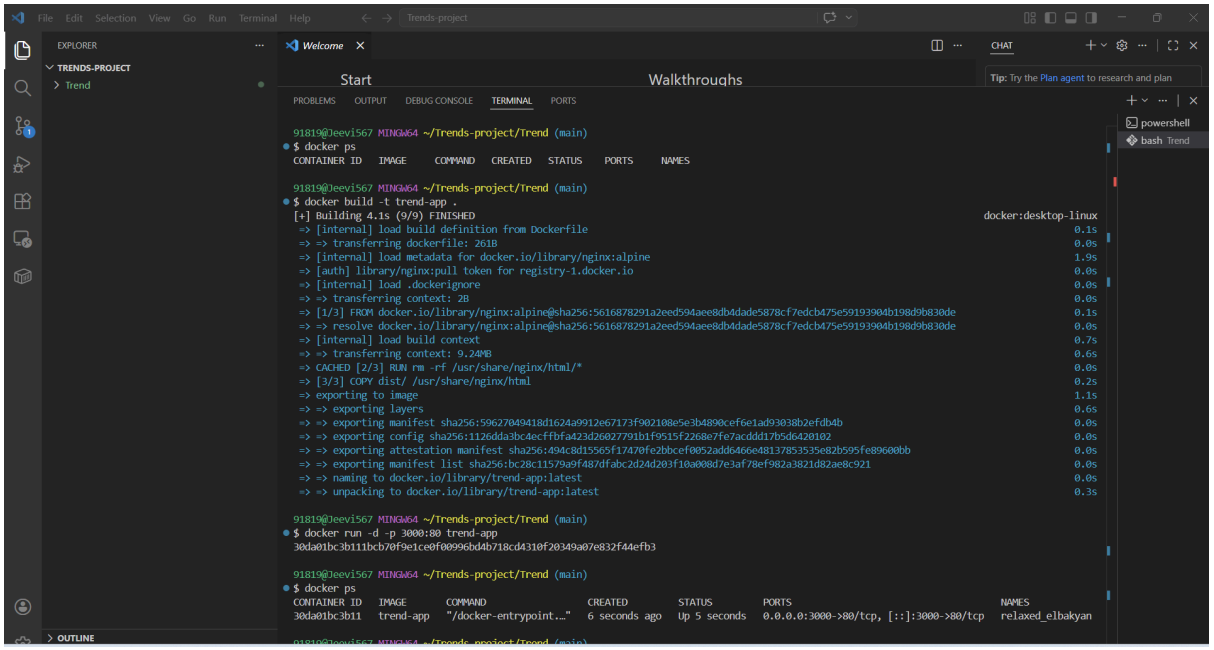


docker run -d -p 3000:80 trend-app

It works:



Inside the container, the app is served on **port 80 by Nginx**, while you access it from your system using **port 3000**.

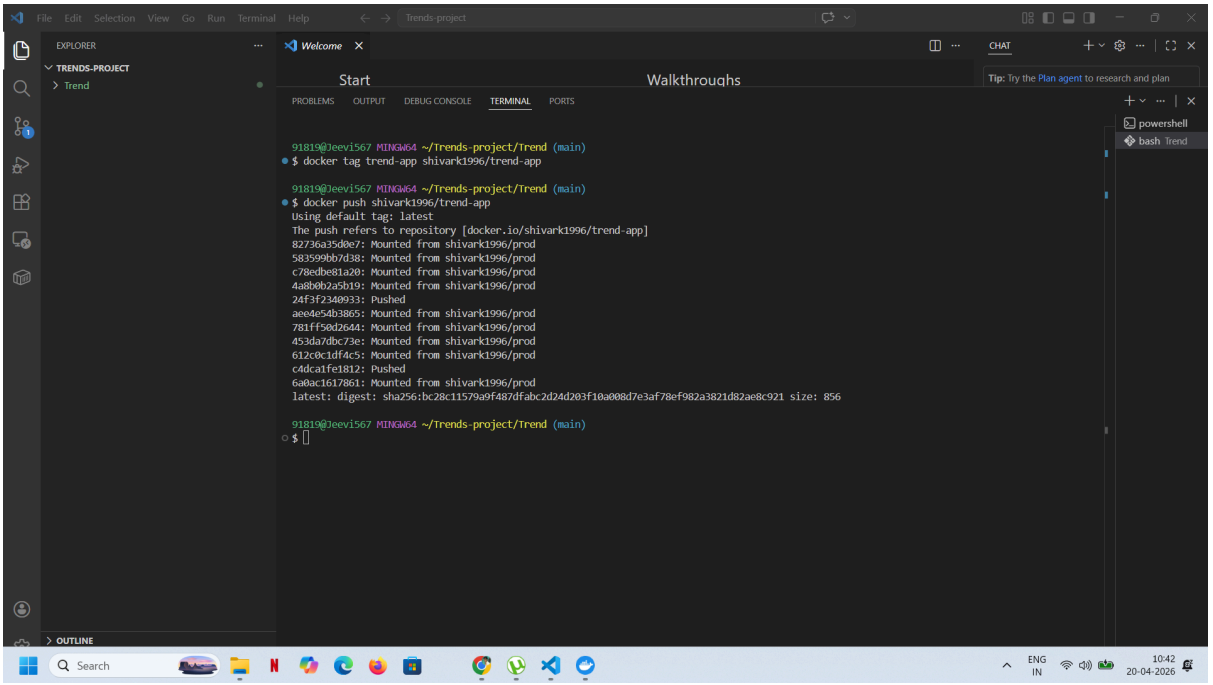


DockerHub:

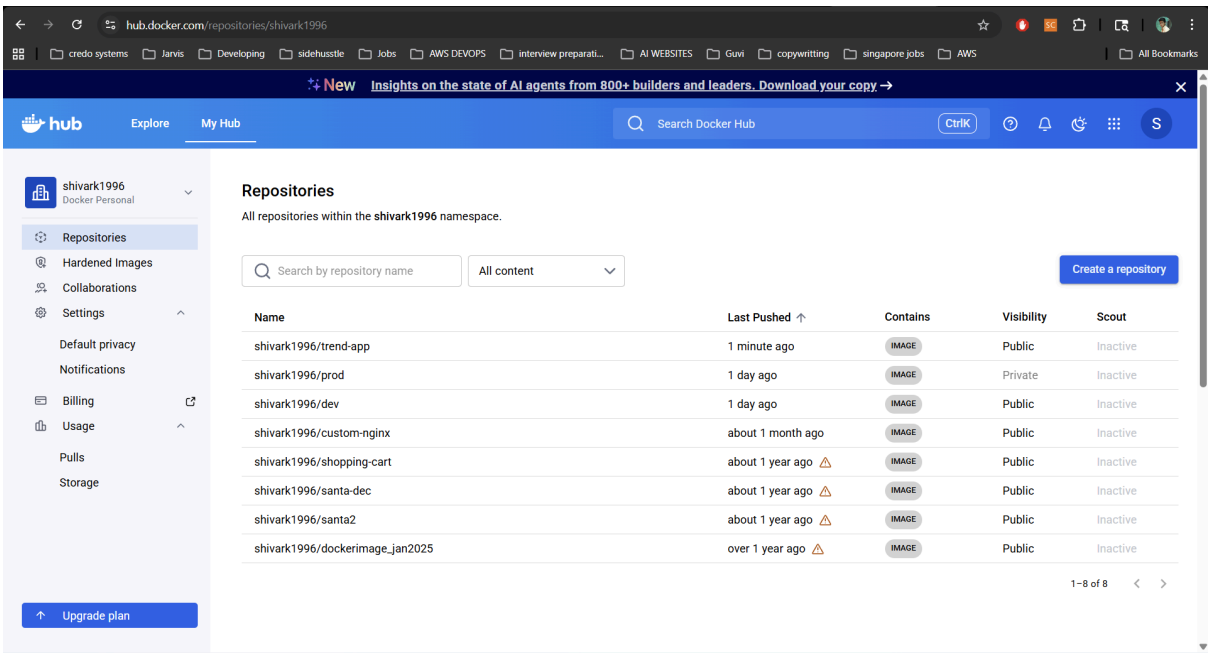
Create a DockerHub repository.

Docker login

Tagging the image and pushing to the docker repository.



docker push shivark1996/trend-app



Terraform:

Define infrastructure in main.tf to create VPC, IAM, EC2 with Jenkins, etc.

Next I am going to create a main.tf to create a infrastructure using this following file

```
91819@Jeevi567 MINGW64 ~/Trends-project
$ ls
Trend/

91819@Jeevi567 MINGW64 ~/Trends-project
$ cd Trend/

91819@Jeevi567 MINGW64 ~/Trends-project/Trend (main)
$ ls
dist/  Dockerfile  README.md  terraform/

91819@Jeevi567 MINGW64 ~/Trends-project/Trend (main)
$ cd terraform/

91819@Jeevi567 MINGW64 ~/Trends-project/Trend/terraform (main)
$ ls
main.tf

91819@Jeevi567 MINGW64 ~/Trends-project/Trend/terraform (main)
$ aws --version
aws-cli/2.34.32 Python/3.14.4 Windows/11 exe/AMD64

91819@Jeevi567 MINGW64 ~/Trends-project/Trend/terraform (main)
$ aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: [redacted]
AWS Secret Access Key [None] [redacted]
Default region name [None]: ap-south-1
Default output format [None]: json

91819@Jeevi567 MINGW64 ~/Trends-project/Trend/terraform (main)
$
```

Inside my main.tf

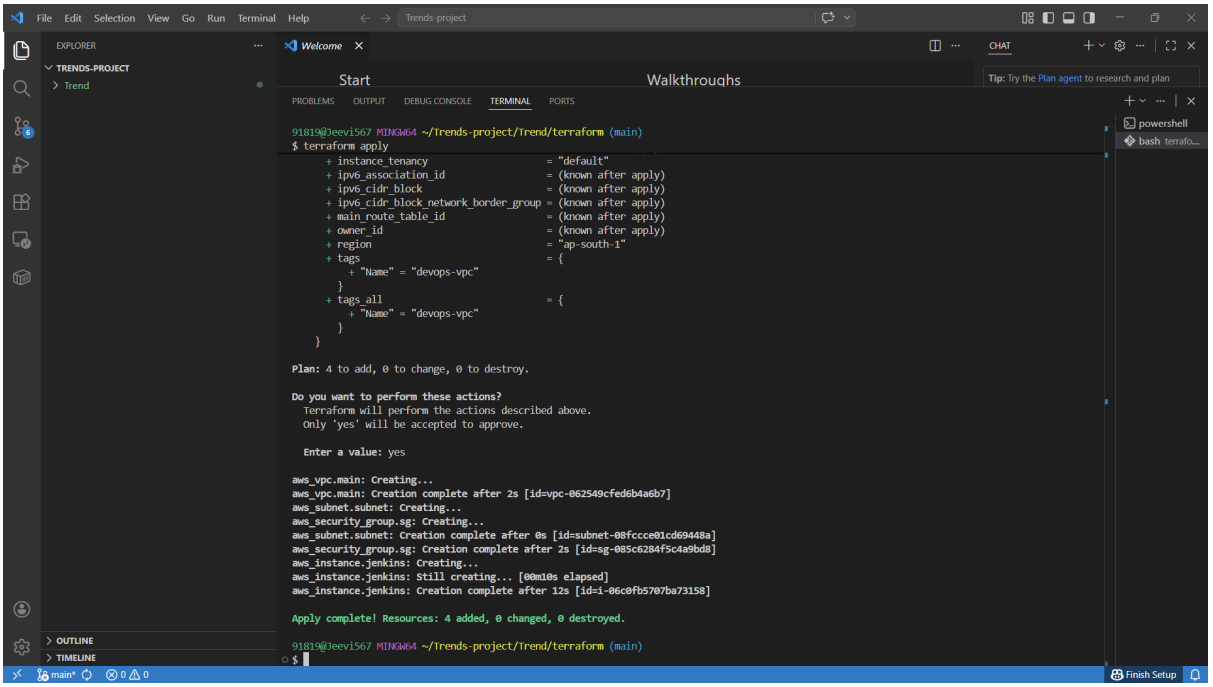
```
91819@Jeevi567 MINGW64 ~/Trends-project/Trend/terraform (main)
$ cat main.tf
provider "aws" {
  region = "ap-south-1"
}

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "devops-vpc"
  }
}

resource "aws_subnet" "subnet" {
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"

  tags = {
    Name = "devops-subnet"
  }
}

resource "aws_security_group" "sg" {
  name   = "devops-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
```

Use terraform command to provision infrastructure.

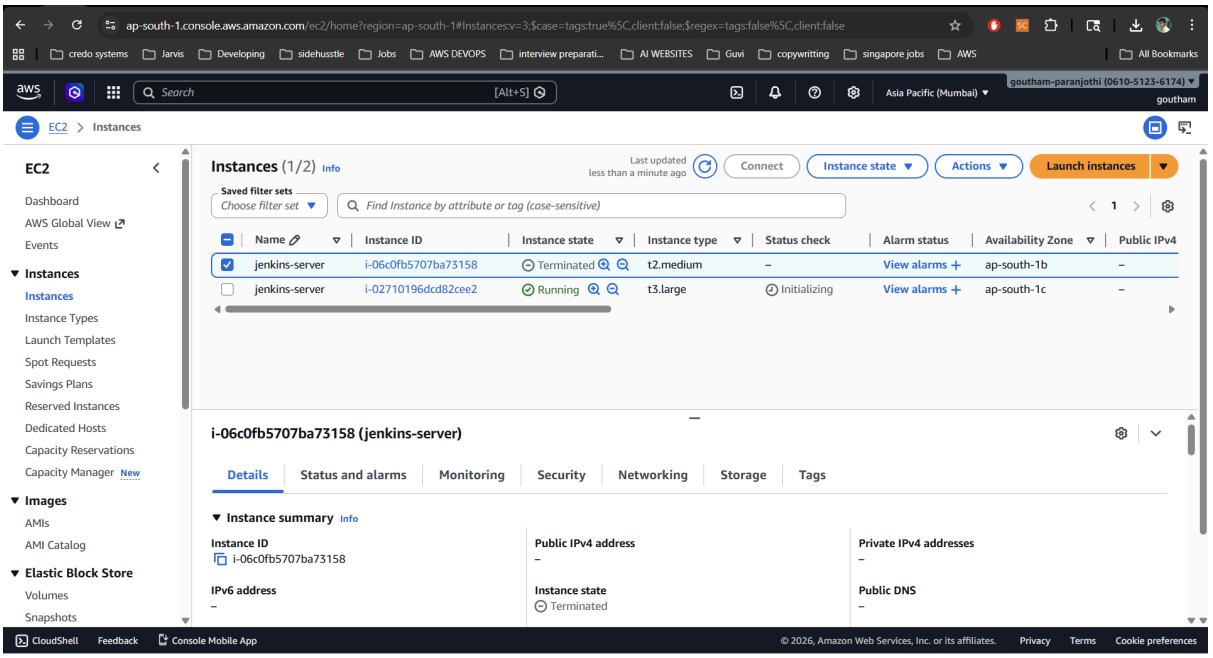
Terraform init

Terraform plan

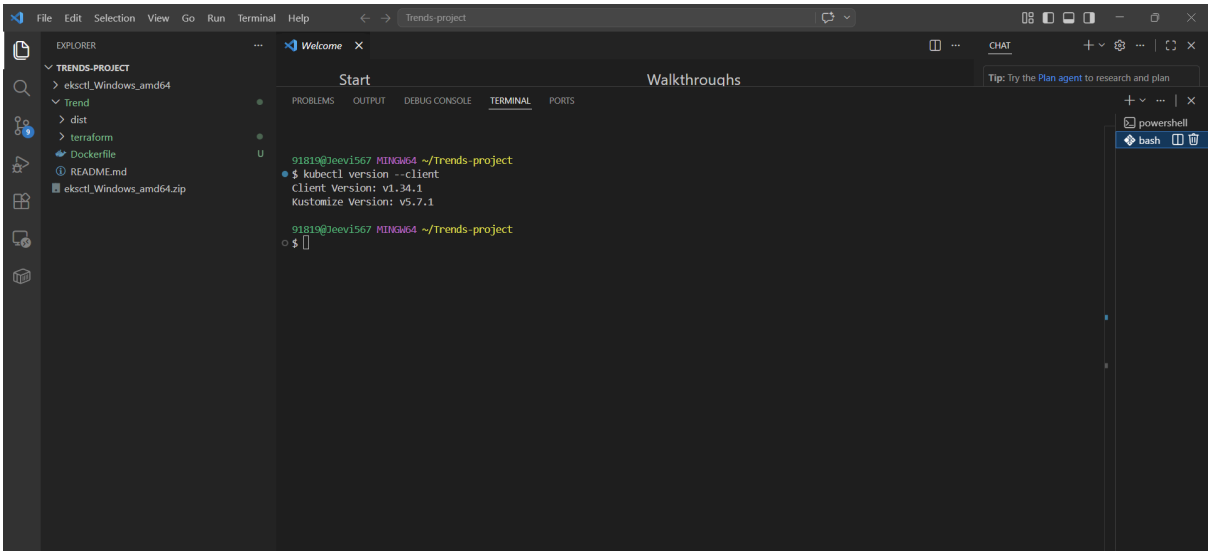
Terraform apply



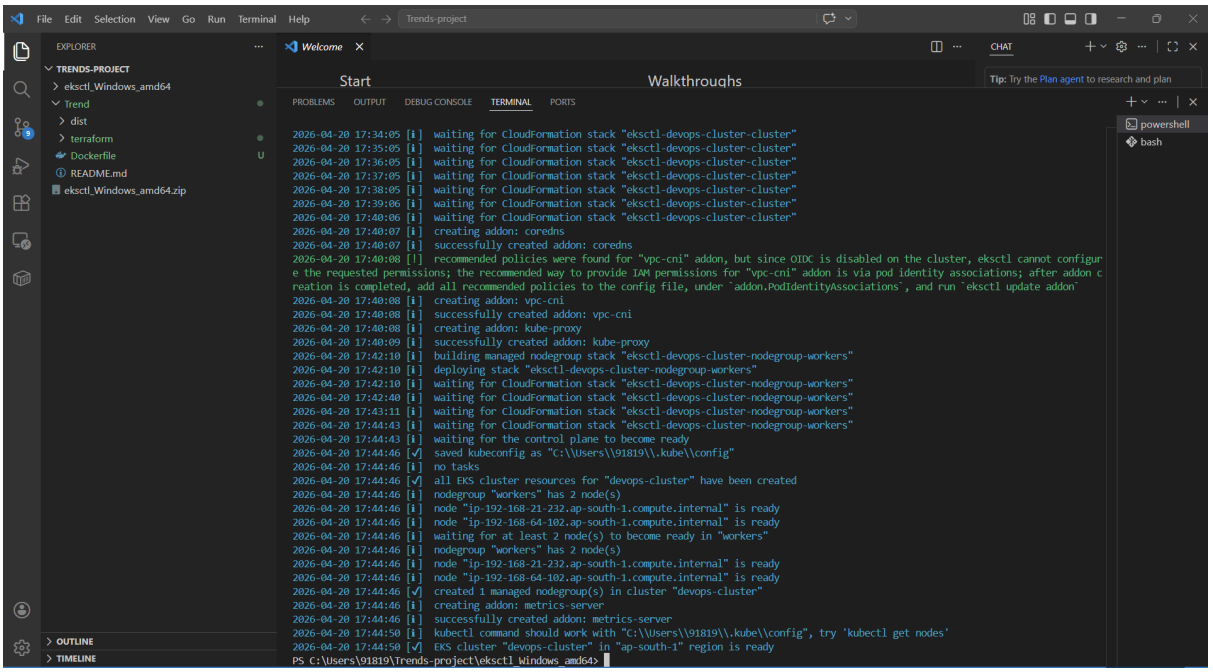
Lets check our terraform code works:



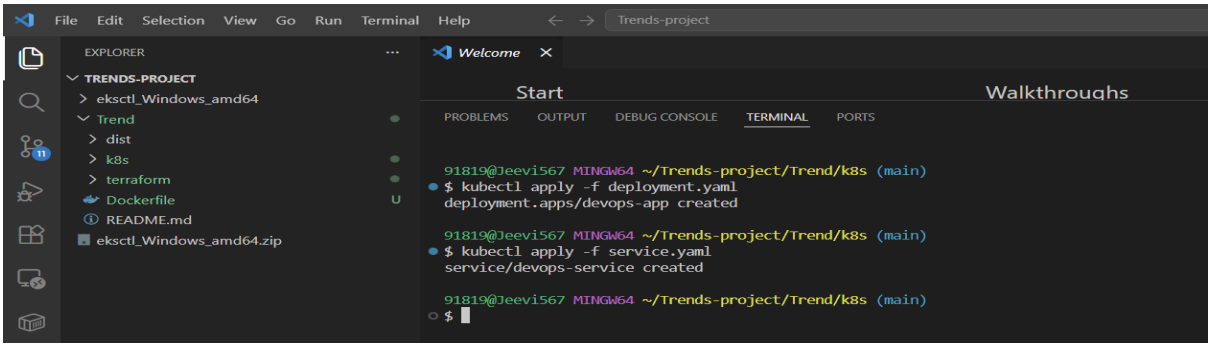
Kubernetes: Setup Kubernetes in AWS EKS and Confirm EKS cluster is running.



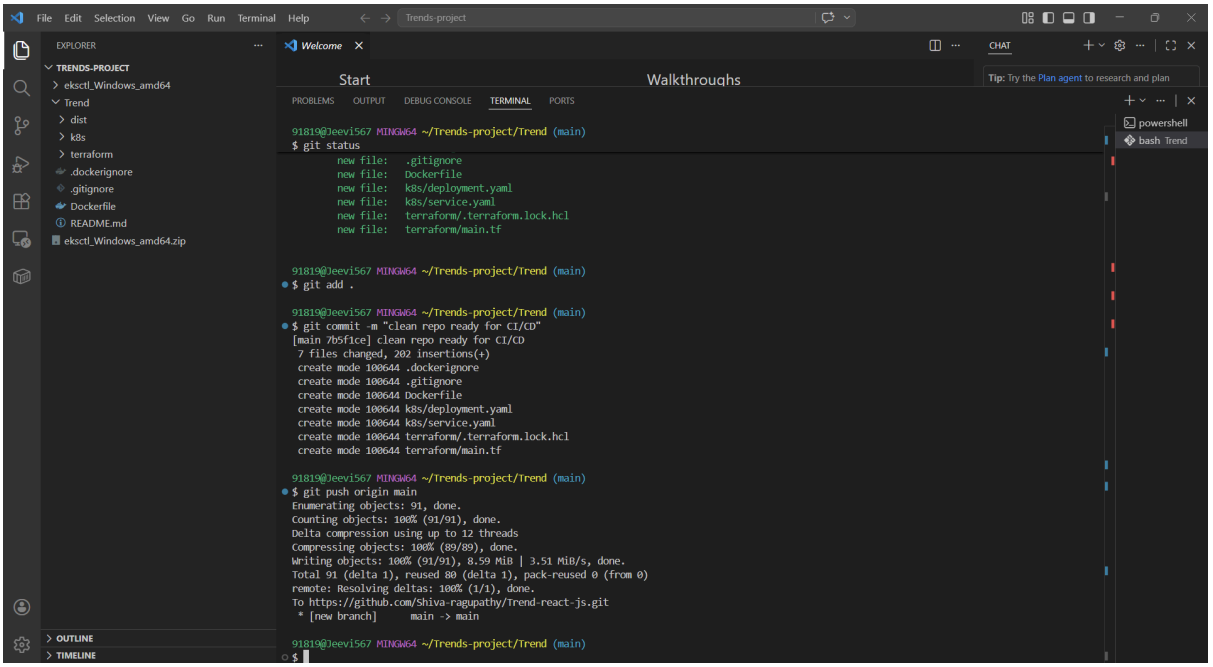
Cluster has been created.



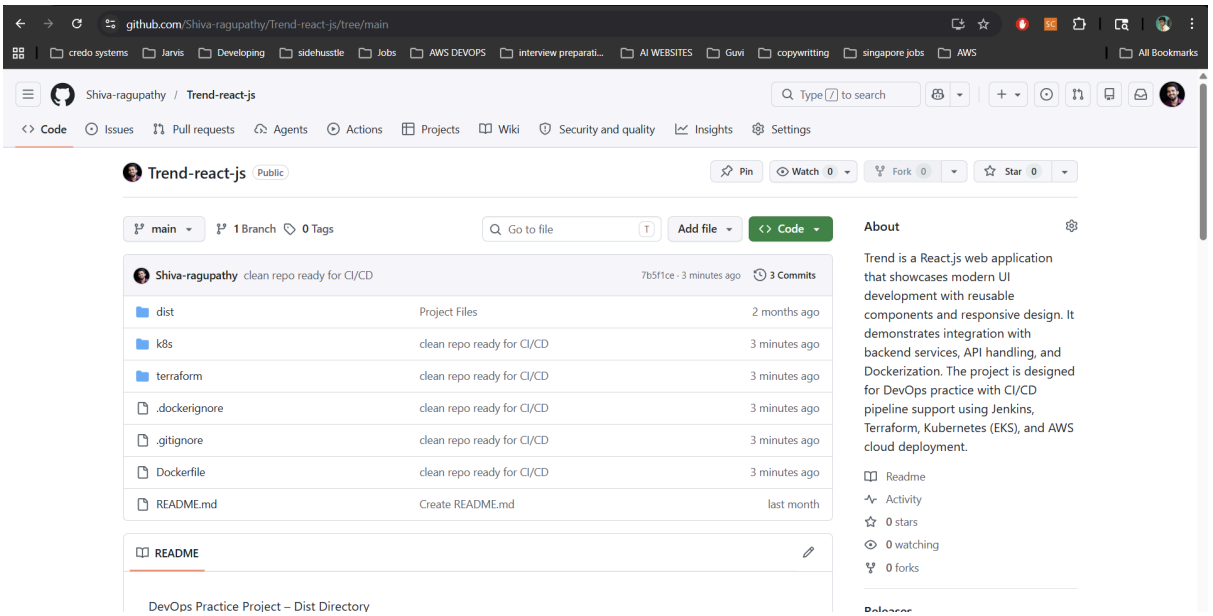
Write deployment and service YAML files.



Version Control: Push the codebase to a Git provider (GitHub).

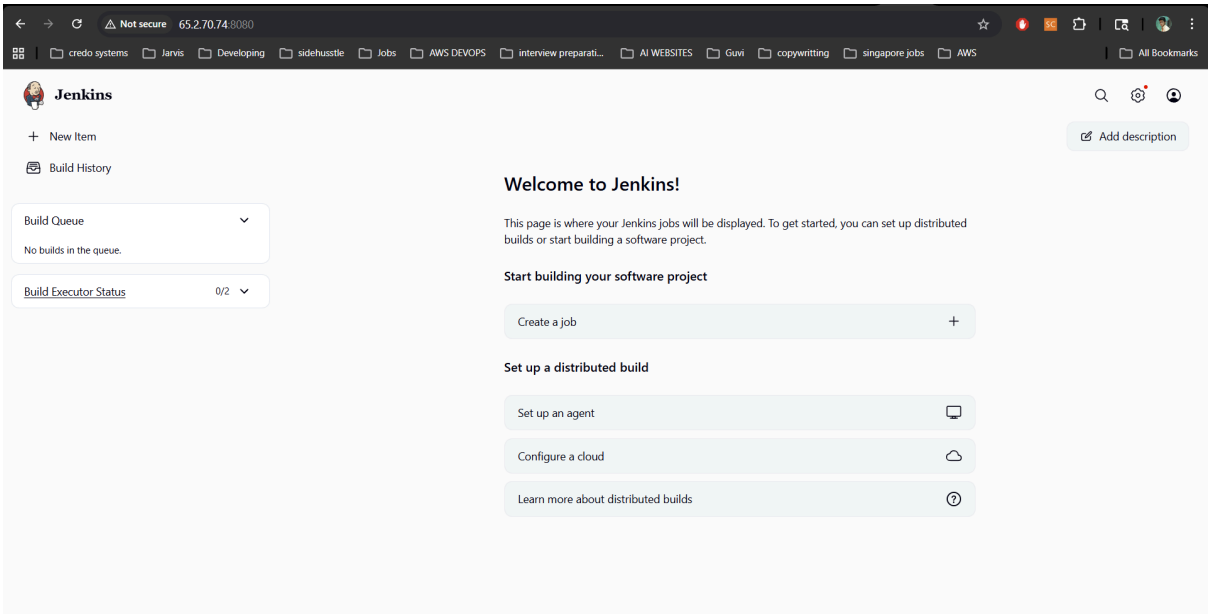


Add gitignore and dockerignore files and use CLI commands to push code.

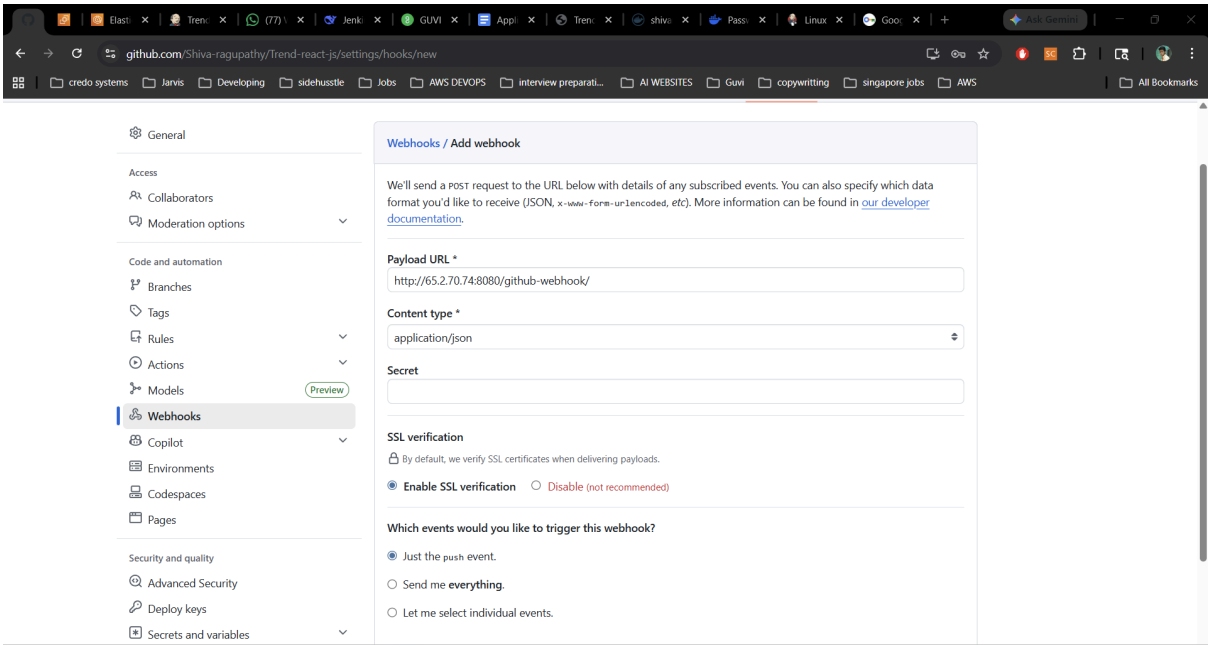


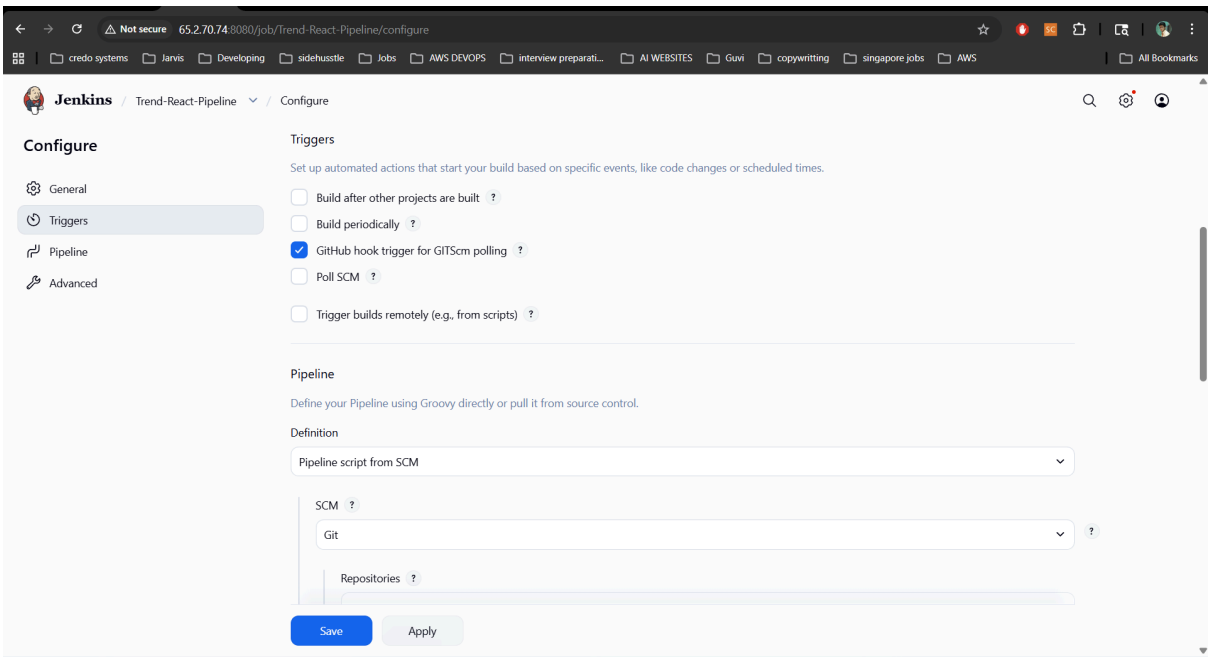
Jenkins:

Install Jenkins and necessary plugins (Docker, Git, Kubernetes, Pipeline) for build, push &deploy applications.

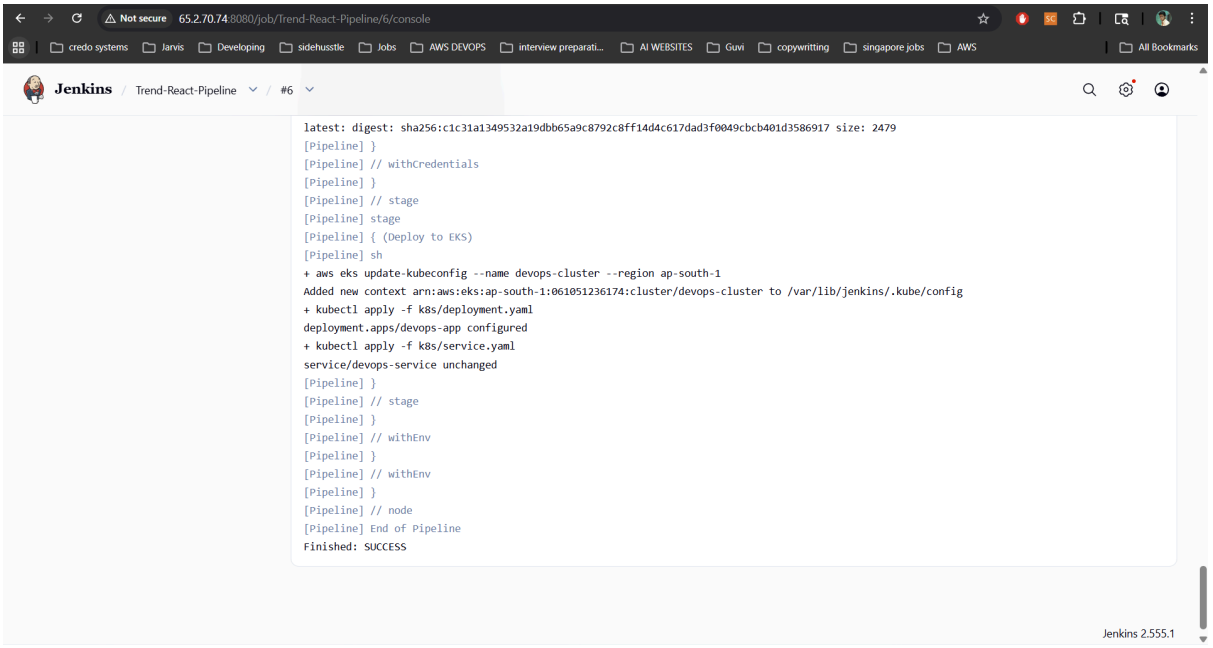


Setup Github and jenkins integration using github webhook build trigger for auto build for every commit.

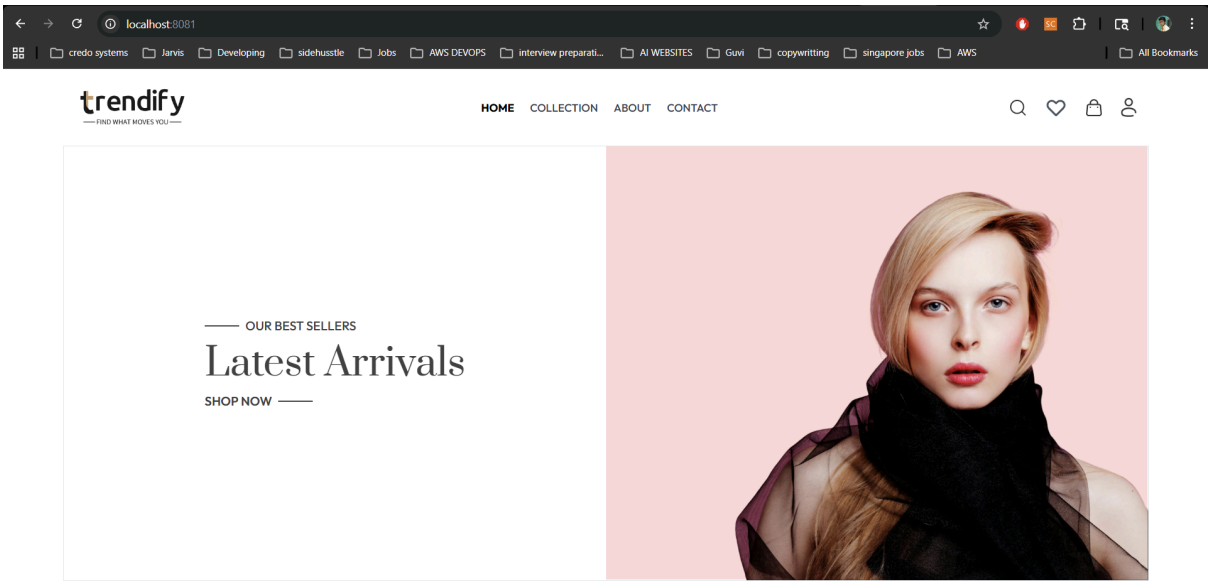




Create a declarative pipeline script and pipeline project to build, push & deploy using CI-CD.

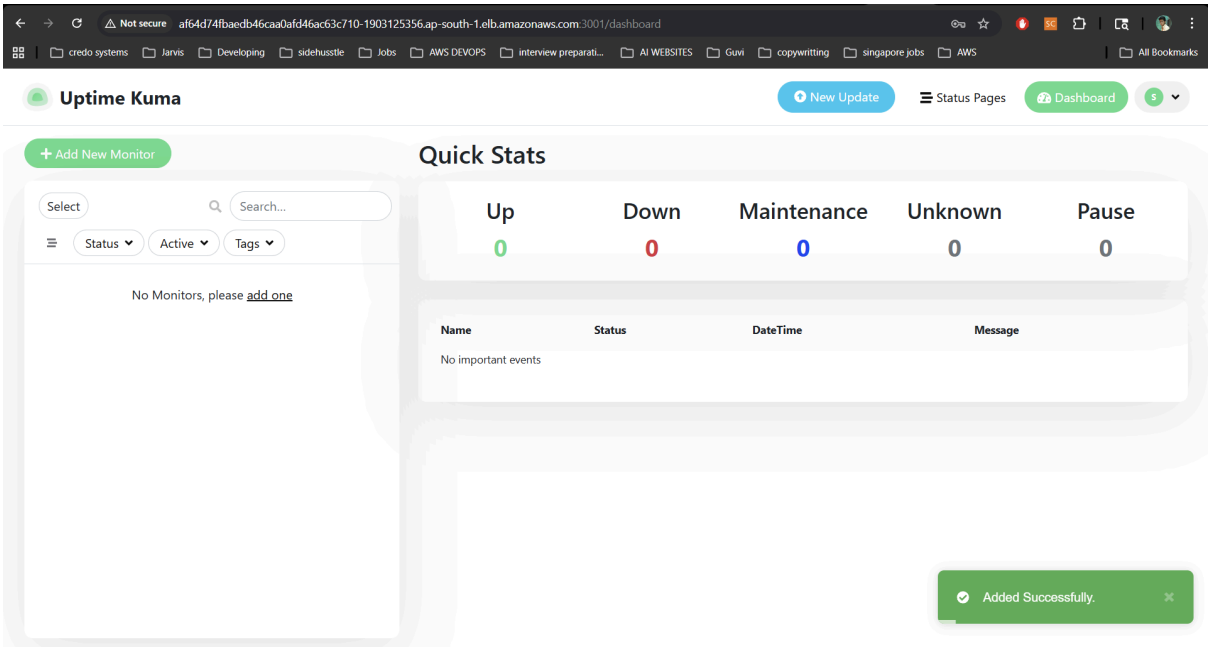


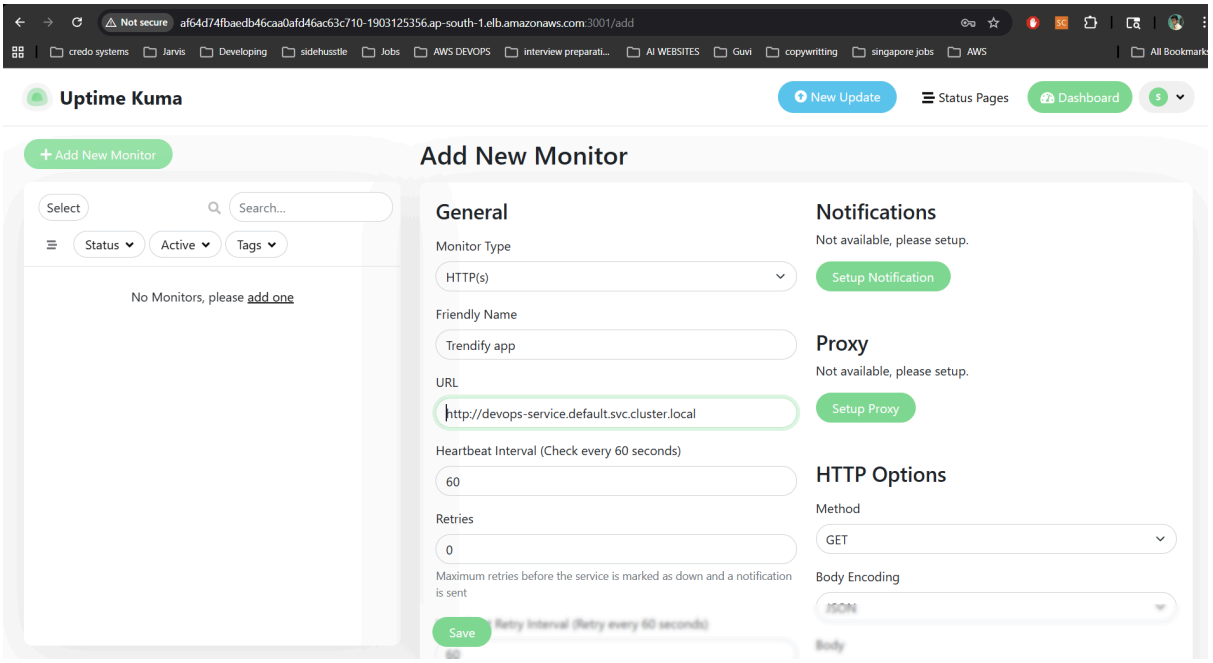
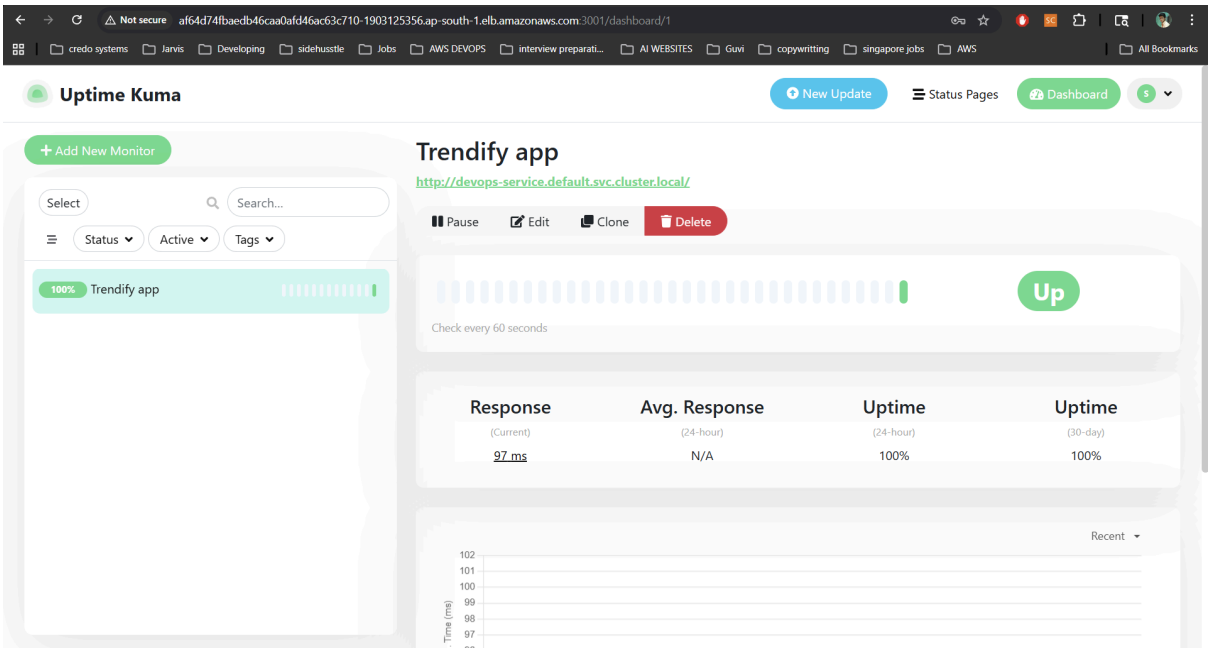
Deploy using kubectl via Jenkins.



Monitoring:

Setup a monitoring system to check the health of the cluster or application (opensource) is highly appreciable.





Task accomplished: